

|                    |                |                 |                       |
|--------------------|----------------|-----------------|-----------------------|
| COURSE CODE/TITLE: | CPE 010        | SECTION:        | CPE21S1               |
| DATE PERFORMED:    | 07-17-26       | DATE SUBMITTED: | 07-17-26              |
| NAME:              | Paula Esguerra | INSTRUCTOR:     | Engr. Jimlord Quejado |

ACTIVITY NO. 2      Arrays Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

```

main.cpp  singly_ll.n
1  #ifndef SINGLY_LL_H
2  #define SINGLY_LL_H
3
4  #include <iostream>
5
6  template <typename T>
7  class SingleList{
8  public:
9      T data; //this contains the data to be stored in the node
10     SingleList<T>* next = nullptr;
11 };
12
13 template <typename T>
14 void SingleListTraversal(SingleList<T>* head) {
15
16     if (head == nullptr) {
17         std::cout << "The list is empty";
18         return;
19     }
20
21     while(head != nullptr) {
22         // PRINT data OF n
23         std::cout<<head->data;
24
25         if(head->next != nullptr) {
26             std::cout << "->";
27         }
28
29         // GO TO NEXT NODE n -> next
30         head = head->next;
31     }
32
33     std::cout << std::endl;

```

```

33     std::cout << std::endl;
34 }
35
36 template <typename T>
37 void sllinsertHead(T newData, SingleList<T>** head) {
38     // 1. Allocate new memory for the new code
39     SingleList<T>* newNode = new SingleList<T>;
40     // 2. Put our data into the new node
41     newNode->data = newData;
42     // 3. Set next to the new node to point to the previous head
43     newNode->next = *head;
44     // 4. Reset head to the new node
45     *head = newNode;
46 }
47
48 template <typename T>
49 void sllGeneralInsert(T newData, SingleList<T>* prevNode) {
50     //1. Check if it is the head node (previous node is null)
51     if(prevNode == nullptr) {
52         //2. If null, print "Previous node cannot be null."
53         std::cout<<"Previous value cannot be null"<<std::endl;
54         return;
55     }
56     //3. Allocate a new node
57     SingleList<T>* newNode = new SingleList<T>;
58     //4. Store data in the new node
59     newNode->data = newData;
60     //5. Point new node to the node previous node was pointing to
61     newNode->next = prevNode->next;
62     //6. Point previous node to the new node
63     prevNode->next = newNode;
64 }

```

```

64 //6. Point previous node to the new node
65 prevNode->next = newNode;
66 }
67
68 template <typename T>
69 void sllInsertEnd(T newData, SingleList<T>** head) {
70 // 1. Allocate a new node
71 SingleList<T>* newNode = new SingleList<T>;
72
73 // 3. Store data in new node
74 newNode->data = newData;
75 // 4. Point to null
76 newNode->next = nullptr;
77
78 // 2. Dereference the head node
79 SingleList<T>* currentNode = *head;
80
81 // 5. Traverse the list
82 while(currentNode->next != nullptr) {
83     currentNode = currentNode->next;
84 }
85
86 // 6. Point the next of the current node to the new node
87 currentNode->next = newNode;
88 }
89
90 template <typename T>
91 void sllDeleteNode(T findData, SingleList<T>** head) {
92
93 // if there is nothing to delete
94 if (*head == nullptr) return;
95
96 SingleList<T>* tempNode = *head;

```

```

95
96     SingleList<T>* currNode = *head;
97     SingleList<T>* prevNode = nullptr;
98
99     // searching the node: 1. the node does not exist + we found data
100    while (currNode != nullptr && currNode->data != findData) {
101        prevNode = currNode;
102        currNode = currNode->next;
103    }
104
105    // if data not found currNode == nullptr in the while loop
106    if (currNode == nullptr) {
107        std::cout << "the data was not found\n" << std::endl;
108        return;
109    }
110
111    // if data was found
112    if (prevNode == nullptr) {
113        *head = currNode->next;
114    } else {
115        prevNode->next = currNode->next;
116    }
117
118    delete currNode;
119 }
120
121
122 template <typename T>
123 void sllDeleteList(SingleList<T>** head) {
124     SingleList<T>* current = *head;
125
126     while (current != nullptr) {
127         SingleList<T>* temp = current;
128         current = current->next;
129         delete temp;
130     }
131
132     *head = nullptr;
133 }
134
135
136
137 #endif

```

```

1  #include <iostream>
2  #include "singly_ll.h"
3
4  int main () {
5      //creation of node PAUE
6
7      //creating a node stored in the stack
8      SingleList<char>* head = nullptr;
9      SingleList<char>* second = nullptr;
10     SingleList<char>* third = nullptr;
11     SingleList<char>* fourth = nullptr;
12
13     //allocate memory in the heap
14     head = new SingleList<char>;
15     second = new SingleList<char>;
16     third = new SingleList<char>;
17     fourth = new SingleList<char>;
18
19     //stored values in the node
20     head ->data = 'P';
21     second ->data = 'A';
22     third ->data = 'U';
23     fourth ->data = 'E';
24
25     //link the nodes
26     head ->next = second;
27     second ->next = third;
28     third ->next = fourth;
29     fourth ->next = nullptr;
30
31     std::cout << "Testing of Traversal: \n";
32     SingleListTraversal(head);
33
34     std::cout << "\n Testing of Traversal: \n";

```

```
34     std::cout << "\nTesting of sllinsertHead\n";
35     sllinsertHead('Q', &head);
36     SingleListTraversal(head);
37
38     std::cout << "\nTesting of sllGeneralInsert\n";
39     sllGeneralInsert('X', head->next->next);
40     SingleListTraversal(head);
41
42     std::cout << "\nTesting of sllInsertEnd\n";
43     sllInsertEnd('W', &head);
44     SingleListTraversal(head);
45
46     std::cout << "\nTesting deleting node\n";
47     sllDeleteNode('Q', &head);
48     SingleListTraversal(head);
49     std::cout << std::endl;
50     sllDeleteNode('X', &head);
51     SingleListTraversal(head);
52     std::cout << std::endl;
53     sllDeleteNode('W', &head);
54     SingleListTraversal(head);
55     std::cout << std::endl;
56
57     std::cout << "Deleting list\n";
58     sllDeleteList(&head);
59     SingleListTraversal(head);
60
61     return 0;
62 }
```

```

C:\Users\Paramount Laptop 5  X + v
Testing of Traversal:
P->A->U->E

Testing of sllinsertHead
Q->P->A->U->E

Testing of sllGeneralInsert
Q->P->A->A->U->E

Testing of sllInsertEnd
Q->P->A->A->U->E->U

Testing deleting node
P->A->A->U->E->U

P->A->U->E->U

the data was not found

P->A->U->E->U

Deleting list
The list is empty
-----
Process exited after 0.1721 seconds with return value 0
Press any key to continue . . . |

```

#### My Conclusion:

The pointers create a pointer variable on the stack frame to hold a memory address of a null; the nodes, on the other hand, use the "new" operator to request memory from the heap at runtime, so the pointer variable is updated to store a new heap address. The arrow operator is used to dereference the pointer and access "data" and "next." By assigning the head node, it will store the heap address to the second node. The head pointer has two methods, or maybe patterns, to pass the value and pass the pointer or reference. Passing the value is to pass a copy of the head address, but the function cannot change where the head points to the main. Passing the pointer or reference, functions like "sllInsertHead" and "sllInsertEnd" require a pointer to a pointer. It will be inserting a new head or can delete the entire list; it requires the function to modify permanently the original head back to the main function. Traversal is stepping through the nodes until hitting a nullptr.

The class is used as a custom data type and structural pointers, like bundled data. The header guards are a construction file, and it can prevent compilation errors if the header is included in multiple files. Double pointers are used to insert and delete, it can permanently modify the head variable inside a function, it must pass the address of the pointer.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

### 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** \_\_\_\_\_

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** \_\_\_\_\_

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** \_\_\_\_\_

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.